

FICHE DE TD 3/3 • DIFFICILE

Informatique • Tle C D E

Écriture et exécution d'algorithmes intégrant une structure de données

Module 3 : Algorithmique et programmation

Consigne : Les exercices suivants portent sur la manipulation de structures de données (enregistrements, tableaux, piles) et l'analyse rigoureuse d'algorithmes de recherche et de tri. Toute trace d'exécution doit être présentée sous forme de tableau complet (ligne, instruction, variables, écran).

EXERCICE 1 Gestion d'un stock de médicaments – Structures composites et tris

On souhaite gérer le stock d'une pharmacie. Chaque médicament est caractérisé par : un code (entier), un nom (chaîne), un prix unitaire (réel) et une quantité en stock (entier). On déclare le type `Médicament` (enregistrement) et un tableau `Stock` de 100 éléments de ce type. La variable `NbMed` (entier) contient le nombre réel de médicaments saisis ($0 < \text{NbMed} \leq 100$).

1. Écrire la déclaration complète du type `Médicament` et du tableau `Stock` en pseudo-code (style MINESEC).
2. Proposer un algorithme `SaisirStock` permettant de remplir le tableau `Stock` pour `NbMed` médicaments. On vérifiera que le code saisi est strictement positif et unique (non déjà présent dans le tableau).
3. Écrire l'algorithme `RechercherCode(CodeRech : Entier) : Entier` réalisant une recherche séquentielle avec drapeau (booléen) du médicament dont le code est `CodeRech`. Il renvoie l'indice de l'élément trouvé ou -1 en cas d'échec.
4. On veut trier le tableau `Stock` par ordre croissant de prix unitaire en utilisant le tri par insertion.
 - (a) Donner l'invariant de boucle de l'algorithme de tri par insertion appliqué au champ `Prix`.
 - (b) Écrire l'algorithme complet `TrierParPrix`.
5. Trace d'exécution : On considère l'état initial suivant pour `NbMed = 5` (seuls les champs `Code` et `Prix` sont pertinents) :

Indice	0	1	2	3	4
Code	105	102	108	101	104
Prix	2500	1200	3100	950	1800

Compléter la table de trace de l'algorithme `TrierParPrix` pour les deux premières itérations de la boucle principale (variable `i` de 1 à 2). On demandera d'afficher les va-

riables i, j, Cle (enregistrement temporaire), et le contenu du tableau `Stock` (codes et prix) après chaque insertion.

EXERCICE 2 Analyse critique et complexité du tri par insertion

On étudie l'algorithme de tri par insertion standard sur un tableau $T[0..N-1]$ d'entiers.

```
Pour i de 1 à N-1 faire
    Cle := T[i]
    j := i - 1
    TantQue (j >= 0) ET (T[j] > Cle) faire
        T[j+1] := T[j]
        j := j - 1
    FinTantQue
    T[j+1] := Cle
FinPour
```

1. Démontrer formellement que l'invariant « Le sous-tableau $T[0..i-1]$ est trié et contient les mêmes éléments que le sous-tableau initial $T[0..i-1]$ » est vérifié à chaque itération de la boucle `Pour`.
2. Déterminer le nombre exact de comparaisons entre éléments ($T[j] > Cle$) effectuées dans le meilleur cas et dans le pire cas. Justifier en fonction de l'état initial du tableau.
3. On exécute cet algorithme sur le tableau $T = [5, 2, 8, 1, 9]$ ($N = 5$).
 - (a) Construire la table de trace complète (variables $i, j, Cle, T[0], T[1], T[2], T[3], T[4]$) jusqu'à l'obtention du tableau trié.
 - (b) Combien d'affectations $T[j+1] := T[j]$ ont été réalisées au total?
4. Proposer une modification de la condition du `TantQue` pour rendre le tri stable (conserver l'ordre relatif des éléments égaux) si on triait des enregistrements sur une clé secondaire. Justifier.

EXERCICE 3 Implémentation d'une Pile par tableau – Structure récursive simulée

Une pile (LIFO) est implémentée statiquement par un tableau $Pile[0..MAX-1]$ d'entiers et un entier `Sommet` indiquant l'indice du dernier élément empilé (`Sommet = -1` si pile vide). `MAX` est une constante fixée à 50.

1. Écrire les algorithmes (fonctions/procédures) suivants en pseudo-code :
 - (a) `EstVide()` : Booléen

- (b) `EstPleine()` : Booléen
 - (c) `Empiler(Valeur : Entier)` – Affiche « Erreur : Pile pleine » si saturée.
 - (d) `Depiler()` : Entier – Renvoie la valeur dépilée ou -1 si vide (après message d'erreur).
2. On utilise cette pile pour vérifier le bon parenthésage d'une expression arithmétique ne contenant que des parenthèses ouvrantes (et fermantes). Écrire l'algorithme `VerifierParentheses(Ch : Chaîne) : Booléen` qui parcourt la chaîne `Ch`, empile les indices des parenthèses ouvrantes et dépile à chaque fermante. L'algorithme renvoie `Vrai` si la pile est vide à la fin et qu'aucune erreur de dépilage sur pile vide n'a eu lieu.
 3. Trace : Appliquer l'algorithme `VerifierParentheses` sur la chaîne `Ch = "(( ) ( ) )"` (longueur 6, indices 0 à 5). Présenter la trace sous forme de tableau avec les colonnes : `IndiceLu`, `CaractereLu`, `Action` (`Empiler/Depiler/Erreur`), `Sommet`, `Contenu Pile[0..Sommet]`. Conclure sur le résultat.

EXERCICE 4 Recherche séquentielle : sentinelle vs drapeau – Problème ouvert

On compare deux variantes de la recherche séquentielle d'une valeur `Val` dans un tableau `T[0..N-1]` d'entiers.

Variante A (Drapeau) :

```
Trouve := Faux ; i := 0
TantQue (i < N) ET (Non Trouve) faire
    Si T[i] = Val Alors Trouve := Vrai
    Sinon i := i + 1
FinTantQue
```

Variante B (Sentinelle) :

```
T[N] := Val // On suppose le tableau dimensionné à N+1
i := 0
TantQue T[i] <> Val faire
    i := i + 1
FinTantQue
Si i < N Alors Trouve := Vrai Sinon Trouve := Faux
```

1. Expliquer pourquoi la Variante B nécessite un tableau de taille $N+1$ et pourquoi elle ne fonctionne pas correctement si `Val` est déjà présent en `T[N-1]` dans le tableau original (avant ajout de la sentinelle). Proposer une correction mineure de l'initialisation pour lever cette ambiguïté.
2. Comparer le nombre d'opérations élémentaires (comparaisons, affectations, tests de boucle) effectuées par les deux variantes dans le cas où `Val` est trouvé à l'indice k ($0 \leq k < N$). Laquelle est théoriquement la plus rapide ? Pourquoi ?

3. Dans quel cas concret (type de données, architecture mémoire, langage) la Variante A serait-elle préférable malgré sa lenteur relative ? Justifier.
4. Écrire un algorithme unique `RechercheTous(T, N, Val, TabIndices, NbOcc)` qui stocke dans `TabIndices` les indices de toutes les occurrences de `Val` dans `T` et met à jour `NbOcc` (nombre d'occurrences). On utilisera une seule boucle `Pour`. Préciser la taille minimale de `TabIndices`.